PANDUAN LENGKAP PENGEMBANGAN APLIKASI PERPUSTAKAAN DIGITAL

Implementasi Operasi CRUD Lengkap, Manajemen Berkas Gambar (Cover), dan Pencarian Multi-Kolom Berbasis Framework Laravel

1. Pendahuluan dan Ringkasan Arsitektur Proyek

Dokumen ini disusun sebagai panduan teknis langkah-demi-langkah (step-by-step) komprehensif untuk membangun aplikasi web pengelolaan perpustakaan digital interaktif dari awal (from scratch). Aplikasi ini mengimplementasikan operasi CRUD (Create, Read, Update, Delete) secara penuh, fitur unggah dan manajemen berkas gambar cover sampul buku, pencarian dinamis multi-kolom, serta penyajian antarmuka responsif bernuansa tema gelap elegan (Dark Maroon Theme) yang terinspirasi dari tampilan katalog buku modern.

Seluruh alur kerja dirancang secara terstruktur dengan mengedepankan standar industri pengembangan web modern, mulai dari pemeriksaan dependensi lingkungan kerja (environment), pembuatan migration basis data, enkapsulasi model Eloquent, perancangan routing resource, pembuatan controller logika bisnis, hingga implementasi antarmuka Blade dengan Tailwind CSS dan Font Awesome.

Fitur Utama Aplikasi yang Dikembangkan:

- **1. Pembuatan Data (Create):** Penyimpanan data buku meliputi judul, penulis, penerbit, tahun terbit, kategori/genre, dan berkas gambar sampul.
- **2. Pembacaan Data (Read):** Penyajian data dalam bentuk grid katalog kartu interaktif pada halaman utama, disertai halaman rincian detail buku secara menyeluruh.
- **3. Pembaruan Data (Update):** Pengubahan data buku disertai fleksibilitas mengganti gambar cover, yang secara otomatis menghapus berkas fisik lama dari direktori penyimpanan penyimpanan internal server guna menghemat memori.
- **4. Penghapusan Data (Delete):** Penghapusan entitas buku dari basis data MySQL yang disertai penghapusan berkas cover terkait secara bersih dan aman.
- **5. Pencarian Multi-Kolom (Search):** Fitur filter pencarian instan pada bilah navigasi atas yang mampu mencari kata kunci pada judul, penulis, penerbit, tahun, maupun kategori secara simultan.
- **6. Antarmuka Tema Gelap:** Tata letak berbasis Tailwind CSS yang responsif, modern, dan ergonomis dengan perpaduan latar belakang marun gelap dan aksen merah menyala.

2. Prasyarat Sistem dan Lingkungan Kerja

Sebelum melangkah ke proses inisialisasi kode, pastikan lingkungan komputer lokal Anda telah memenuhi spesifikasi teknis berikut:

Komponen	Versi Minimum	Keterangan Fungsi
PHP	Versi 8.1 / 8.2 ke atas	Runtime bahasa pemrograman backend utama.
Composer	Versi 2.x	Pengelola dependensi pustaka dan paket PHP resmi.
MySQL Database	Versi 5.7 / 8.0 / MariaDB	Sistem manajemen basis data relasional (RDBMS).
Ekstensi PHP	Fileinfo, PDO, Mbstring, OpenSSL	Dibutuhkan untuk validasi berkas upload dan kriptografi.
Web Browser	Google Chrome / Firefox / Edge	Untuk menjalankan dan menguji antarmuka web klien.

3. Langkah 1: Pemeriksaan Lingkungan dan Pembuatan Proyek Baru

Langkah awal adalah memastikan bahwa executable PHP dan Composer sudah terdaftar pada variabel lingkungan (Environment Variables / PATH) sistem operasi Anda.

Buka aplikasi Terminal, Command Prompt (CMD), atau PowerShell, lalu jalankan perintah berikut untuk memeriksa versinya:

Perintah 1.1: Memeriksa Versi PHP dan Composer

```
php -v
composer -v
```

Apabila kedua perintah di atas menampilkan informasi nomor versi secara valid, lanjutkan dengan membuat proyek Laravel baru bernama 'buku' menggunakan perintah Composer:

Perintah 1.2: Pembuatan Proyek Laravel Baru

```
composer create-project laravel/laravel buku
```

Setelah proses pembuatan selesai dan dependensi berhasil diunduh, masuklah ke dalam folder direktori proyek yang baru saja dibuat:

Perintah 1.3: Berpindah ke Direktori Proyek

```
cd buku
```

> **Penting:** *Pastikan seluruh eksekusi perintah berikutnya (seperti php artisan) selalu dilakukan di dalam direktori root 'buku' ini.*

4. Langkah 2: Konfigurasi Basis Data MySQL dan Berkas Environment

Sebelum menghubungkan aplikasi ke database, buatlah basis data baru di MySQL. Jika Anda menggunakan panel phpMyAdmin, buka http://localhost/phpmyadmin melalui browser, klik menu 'Databases', beri nama basis data dengan db_library, lalu klik 'Create'.

Selanjutnya, buka proyek 'buku' di text editor pilihan Anda (misalnya Visual Studio Code). Buka berkas bernama .env yang berada di folder root proyek, lalu temukan baris konfigurasi basis data dan sesuaikan nilainya sebagai berikut:

Konfigurasi 2.1: Pengaturan Basis Data pada Berkas .env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_library
DB_USERNAME=root
DB_PASSWORD=
```

Penjelasan parameter konfigurasi:

- **DB_CONNECTION:** Menentukan driver basis data relasional yang digunakan, dalam hal ini MySQL.
- **DB_HOST & DB_PORT:** Alamat peladen (host) server database lokal Anda.
- **DB_DATABASE:** Nama skema database yang telah dibuat sebelumnya.
- **DB_USERNAME & DB_PASSWORD:** Kredensial autentikasi pengguna MySQL (pada instalasi bawaan XAMPP, username adalah 'root' dan password dibiarkan kosong).

5. Langkah 3: Konfigurasi Tautan Penyimpanan Gambar (Storage Link)

Laravel secara standar menyimpan berkas unggahan publik ke dalam direktori storage/app/public/ demi keamanan dan isolasi berkas. Namun, direktori ini tidak dapat diakses langsung oleh peramban klien melalui protokol HTTP.

Untuk menjembatani direktori penyimpanan tersebut agar dapat diakses publik, kita wajib membuat symbolic link (tautan simbolis) dari folder storage/app/public ke folder public/storage.

Jalankan perintah Artisan berikut di terminal Anda:

Perintah 3.1: Membuat Symbolic Link Storage

```
php artisan storage:link
```

> **Catatan Konfigurasi Storage:** *Jika perintah ini berhasil, Anda akan menerima pesan konfirmasi '[public/storage] has been linked to [storage/app/public]'. Seluruh gambar cover yang disimpan nanti akan otomatis dapat dimuat oleh tag <img> di halaman web.*

6. Langkah 4: Pembuatan Model Eloquent, Migration, dan Controller

Laravel menyediakan flag perintah artisan yang memungkinkan pengembang untuk membuat berkas Model, berkas Migration skema tabel, dan berkas Resource Controller sekaligus dalam satu baris perintah.

Eksekusi perintah berikut di terminal:

Perintah 4.1: Pembuatan Model, Migration, dan Controller

```
php artisan make:model Book -mcr
```

Rincian fungsi flag perintah di atas:

- **Book:** Membuat berkas model app/Models/Book.php untuk berinteraksi dengan tabel database.
- **-m (Migration):** Menghasilkan berkas migrasi database di folder database/migrations/ untuk mendefinisikan kolom tabel books.

- **-c (Controller):** Menghasilkan berkas controller di app/Http/Controllers/BookController.php.
- **-r (Resource):** Menginstruksikan Laravel untuk menyusun controller lengkap dengan 7 method resource standar (index, create, store, show, edit, update, destroy).

7. Langkah 5: Perancangan Skema Database (Migration)

Buka berkas migrasi yang baru saja digenerate di dalam folder database/migrations/xxxx_xx_xx_xxxxxx_create_books_table.php. Temukan method up() dan sesuaikan kolom tabel books agar menampung seluruh atribut buku yang diperlukan:

Kode 5.1: File Migration database/migrations/xxxx_create_books_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('books', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->string('author');
            $table->string('publisher');
            $table->integer('year');
            $table->string('category');
            $table->string('cover')->nullable();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('books');
    }
```

```
};
```

Struktur rincian kolom tabel 'books':

Nama Kolom	Tipe Data	Sifat / Keterangan
id	BIGINT UNSIGNED	Kunci utama (Primary Key) auto increment.
title	VARCHAR(255)	Judul buku (Wajib diisi).
author	VARCHAR(255)	Nama pengarang/penulis buku (Wajib diisi).
publisher	VARCHAR(255)	Nama badan penerbit buku (Wajib diisi).
year	INTEGER	Tahun terbit buku dalam format angka tahun.
category	VARCHAR(255)	Kategori / genre buku (cth: Misteri, Fiksi).
cover	VARCHAR(255)	Jalur path file gambar di storage (Boleh kosong/nullable).
timestamps	TIMESTAMP	Mencatat created_at dan updated_at secara otomatis.

Setelah menyimpan berkas migrasi tersebut, terapkan skema tabel ke dalam database MySQL Anda dengan menjalankan perintah:

Perintah 5.2: Menjalankan Migrasi ke Database

```
php artisan migrate
```

8. Langkah 6: Konfigurasi Mass Assignment pada Model Eloquent

Fitur Mass Assignment Protection pada Laravel mencegah pengguna luar menyuntikkan data ke kolom-kolom yang tidak diinginkan. Kita wajib mendeklarasikan atribut properti $fillable pada kelas model agar method create() dan update() dapat mengisi kolom secara aman.

Buka file app/Models/Book.php dan perbarui kodenya menjadi:

Kode 6.1: Model Eloquent app/Models/Book.php

```
<?php

namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'title',
        'author',
        'publisher',
        'year',
        'category',
        'cover',
    ];
}
```

9. Langkah 7: Pembuatan Routing Web Resource

Routing bertugas memetakan URL yang diminta oleh pengguna di peramban ke aksi method controller yang bersangkutan. Dengan menggunakan resource route, Laravel secara otomatis memetakan tujuh endpoint standar RESTful.

Buka berkas routes/web.php, hapus rute bawaan, lalu ganti dengan definisi rute berikut:

Kode 7.1: Berkas Routing routes/web.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\BookController;

/*
|--------------------------------------------------------------------------
| Web Routes
|--------------------------------------------------------------------------
*/
```

```
// Mengarahkan halaman utama (root) langsung ke daftar buku
Route::redirect('/', '/books');

// Resource route untuk seluruh operasi CRUD tabel buku
Route::resource('books', BookController::class);
```

Tabel Pemetaan Rute Resource Controller Laravel:

HTTP Verb	URI Path	Action Controller	Nama Route
GET	/books	BookController@index	books.index
GET	/books/create	BookController@create	books.create
POST	/books	BookController@store	books.store
GET	/books/{book}	BookController@show	books.show
GET	/books/{book}/edit	BookController@edit	books.edit
PUT / PATCH	/books/{book}	BookController@update	books.update
DELETE	/books/{book}	BookController@destroy	books.destroy

10. Langkah 8: Implementasi Logika Bisnis & Validasi CRUD di Controller

Berkas Controller bertanggung jawab mengelola alur data antara Model dan View. Di sini kita menanamkan logika validasi ketat, pengelolaan upload berkas cover, penghapusan berkas cover usang dari storage, serta kemampuan pencarian multi-kolom.

Buka berkas app/Http/Controllers/BookController.php dan isi dengan kode lengkap berikut:

Kode 8.1: File Controller app/Http/Controllers/BookController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class BookController extends Controller
{
```

```
/**
 * Menampilkan daftar seluruh buku beserta fitur pencarian multi-kolom.
 */
public function index(Request $request)
{
    $search = $request->query('search');

    // Filter pencarian pada kolom title, author, publisher, year, atau category
    $books = Book::when($search, function ($query, $search) {
        $query->where('title', 'like', "%{$search}%")
              ->orWhere('author', 'like', "%{$search}%")
              ->orWhere('publisher', 'like', "%{$search}%")
              ->orWhere('year', 'like', "%{$search}%")
              ->orWhere('category', 'like', "%{$search}%");
    })->latest()->get();

    return view('books.index', compact('books', 'search'));
}

/**
 * Menampilkan halaman formulir tambah buku baru.
 */
public function create()
{
    return view('books.create');
}

/**
 * Menyimpan buku baru ke dalam database dan menangani upload gambar cover.
 */
public function store(Request $request)
{
    $validated = $request->validate([
        'title'     => 'required|string|max:255',
        'author'    => 'required|string|max:255',
        'publisher' => 'required|string|max:255',
        'year'      => 'required|numeric',
        'category'  => 'required|string|max:255',
        'cover'     => 'nullable|image|mimes:jpeg,png,jpg,webp|max:2048',
    ]);

    // Simpan file cover ke storage/app/public/covers jika diunggah
    if ($request->hasFile('cover')) {
        $validated['cover'] = $request->file('cover')->store('covers', 'public');
    }

    Book::create($validated);
```

```
    return redirect()->route('books.index')->with('success', 'Buku berhasil ditambahkan!');
}

/**
 * Menampilkan detail informasi lengkap satu buku.
 */
public function show(Book $book)
{
    return view('books.show', compact('book'));
}

/**
 * Menampilkan formulir edit data buku.
 */
public function edit(Book $book)
{
    return view('books.edit', compact('book'));
}

/**
 * Memperbarui data buku dan mengganti gambar cover jika terdapat unggahan baru.
 */
public function update(Request $request, Book $book)
{
    $validated = $request->validate([
        'title'     => 'required|string|max:255',
        'author'    => 'required|string|max:255',
        'publisher' => 'required|string|max:255',
        'year'      => 'required|numeric',
        'category'  => 'required|string|max:255',
        'cover'     => 'nullable|image|mimes:jpeg,png,jpg,webp|max:2048',
    ]);

    if ($request->hasFile('cover')) {
        // Hapus cover lama dari penyimpanan fisik jika ada
        if ($book->cover && Storage::disk('public')->exists($book->cover)) {
            Storage::disk('public')->delete($book->cover);
        }
        // Simpan cover yang baru diunggah
        $validated['cover'] = $request->file('cover')->store('covers', 'public');
    }

    $book->update($validated);

    return redirect()->route('books.index')->with('success', 'Data buku berhasil diperbarui!');
}
```

```
    /**
     * Menghapus data buku dan membersihkan berkas cover dari disk storage.
     */
    public function destroy(Book $book)
    {
        // Hapus file cover fisik dari disk
        if ($book->cover && Storage::disk('public')->exists($book->cover)) {
            Storage::disk('public')->delete($book->cover);
        }

        $book->delete();

        return redirect()->route('books.index')->with('success', 'Buku berhasil dihapus!');
    }
}
```

11. Langkah 9: Pembuatan Antarmuka Berbasis Blade & Tailwind CSS

Untuk menyajikan tampilan visual bertema gelap elegan (Dark Maroon Theme) yang sesuai dengan gambar referensi desain perpustakaan, kita memanfaatkan Tailwind CSS melalui CDN dan pustaka ikon Font Awesome. Seluruh berkas view ditempatkan pada folder resources/views/.

9.1 Master Layout (resources/views/layouts/app.blade.php)

Buat berkas app.blade.php di dalam folder baru resources/views/layouts/. Master layout ini menyajikan struktur sidebar navigasi di sisi kiri dan konten dinamis di sisi kanan.

Kode 9.1: Master Layout resources/views/layouts/app.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Perpustakaan Digital</title>
    <!-- Tailwind CSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>
    <!-- Font Awesome Icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
</head>
<body class="bg-[#120608] text-white flex min-h-screen font-sans">
```

```
    <!-- Sidebar Kiri -->
    <aside class="w-64 bg-[#1a080c] border-r border-red-950/40 p-6 flex flex-col gap-6">
        <div class="flex items-center gap-3 text-xl font-bold text-gray-200">
            <i class="fa-solid fa-book-bookmark text-[#e02438]"></i>
            <span>Perpustakaan</span>
        </div>
        <nav>
            <a href="{{ route('books.index') }}"
               class="flex items-center gap-3 px-4 py-2.5 bg-[#e02438] rounded-xl font-semibold text-white
shadow-md transition hover:bg-red-700">
                <i class="fa-solid fa-book-open"></i>
                <span>Buku</span>
            </a>
        </nav>
    </aside>

    <!-- Area Konten Utama -->
    <main class="flex-1 p-8 overflow-y-auto">
        @yield('content')
    </main>

</body>
</html>
```

9.2 Tampilan Katalog / Daftar Buku (resources/views/books/index.blade.php)

Buat berkas index.blade.php di dalam folder resources/views/books/. Halaman ini menyajikan bilah pencarian responsif, tombol tambah buku, dan grid kartu buku lengkap beserta gambar sampul dan tombol aksi edit serta hapus.

Kode 9.2: Tampilan Katalog resources/views/books/index.blade.php

```
@extends('layouts.app')

@section('content')
<div class="space-y-6">
    <!-- Header Atas: Judul, Bilah Pencarian, dan Tombol Tambah -->
    <div class="flex flex-wrap items-center justify-between gap-4">
        <div class="flex items-center gap-3 text-2xl font-bold">
            <i class="fa-solid fa-book-open text-[#e02438]"></i>
            <h2>Daftar Buku</h2>
        </div>

        <!-- Form Pencarian Multi-Kolom -->
        <form action="{{ route('books.index') }}" method="GET" class="flex items-center gap-2">
```

```
        <input type="text" name="search" value="{{ $search }}"
               placeholder="Cari judul, penulis, penerbit, tahun, kategori..."
               class="w-80 md:w-96 px-4 py-2 bg-white text-gray-900 rounded-lg text-sm focus:outline-none
shadow">
        <button type="submit" class="px-4 py-2 bg-[#e02438] hover:bg-red-700 rounded-lg text-white
transition">
            <i class="fa-solid fa-magnifying-glass"></i>
        </button>
      </form>

      <!-- Tombol Tambah Buku Baru -->
      <a href="{{ route('books.create') }}"
        class="px-4 py-2 bg-[#e02438] hover:bg-red-700 rounded-lg font-semibold flex items-center gap-2
text-sm shadow transition">
        <i class="fa-solid fa-plus"></i> Tambah Buku
      </a>
    </div>

    <!-- Grid Kartu Katalog Buku -->
    <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-5 gap-6">
      @forelse($books as $book)
        <div class="bg-[#1a080c] rounded-2xl overflow-hidden border border-red-950/40 flex flex-col
justify-between shadow-lg hover:border-red-800/60 transition">
          <!-- Cover Sampul Buku -->
          <a href="{{ route('books.show', $book) }}" class="block overflow-hidden">
            @if($book->cover)
              <img src="{{ asset('storage/' . $book->cover) }}" alt="{{ $book->title }}"
                   class="w-full h-72 object-cover hover:scale-105 transition duration-300">
            @else
              <div class="w-full h-72 bg-gray-900 flex items-center justify-center text-gray-500 text-sm">
                <i class="fa-solid fa-image text-3xl"></i>
              </div>
            @endif
          </a>

          <!-- Rincian Informasi Buku -->
          <div class="p-4 space-y-2 flex-1 flex flex-col justify-between">
            <div>
              <span class="inline-block bg-[#e02438] text-[11px] font-semibold px-2.5 py-0.5 rounded-
full mb-2">
                {{ $book->category }}
              </span>
              <h3 class="font-bold text-base text-gray-100 line-clamp-1" title="{{ $book->title }}">
                {{ $book->title }}
              </h3>
              <p class="text-xs text-gray-400 mt-1">{{ $book->author }}</p>
              <p class="text-xs text-gray-400">{{ $book->publisher }} • {{ $book->year }}</p>
            </div>
```

```
                <!-- Tombol Aksi Edit & Hapus -->
                <div class="flex gap-2 pt-3">
                    <a href="{{ route('books.edit', $book) }}"
                       class="flex-1 py-1.5 bg-[#f5b301] hover:bg-amber-500 rounded-lg text-center text-black
font-semibold text-xs flex items-center justify-center transition">
                        <i class="fa-solid fa-pen"></i>
                    </a>
                    <form action="{{ route('books.destroy', $book) }}" method="POST" class="flex-1"
                          onsubmit="return confirm('Apakah Anda yakin ingin menghapus buku ini?')">
                        @csrf
                        @method('DELETE')
                        <button type="submit"
                                class="w-full py-1.5 bg-[#e02438] hover:bg-red-700 rounded-lg text-white font-
semibold text-xs flex items-center justify-center transition">
                            <i class="fa-solid fa-trash"></i>
                        </button>
                    </form>
                </div>
            </div>
        </div>
    @empty
        <div class="col-span-full py-16 text-center text-gray-400 space-y-2">
            <i class="fa-solid fa-folder-open text-4xl text-gray-600"></i>
            <p>Belum ada koleksi buku yang tersedia.</p>
        </div>
    @endforelse
  </div>
</div>
@endsection
```

9.3 Tampilan Detail Rincian Buku (resources/views/books/show.blade.php)

Buat berkas show.blade.php di resources/views/books/. Halaman ini menyajikan cover beresolusi tinggi di sisi kiri dan tabel rincian metadata di sisi kanan, identik dengan screenshot tampilan detail.

Kode 9.3: Detail Buku resources/views/books/show.blade.php

```
@extends('layouts.app')

@section('content')
<div class="space-y-6">
  <div class="flex items-center justify-between">
    <div class="flex items-center gap-3 text-2xl font-bold">
```

```
            <i class="fa-solid fa-book-open text-[#e02438]"></i>
            <h2>Detail Buku</h2>
        </div>
        <a href="{{ route('books.index') }}"
          class="px-4 py-2 bg-[#2a1015] hover:bg-red-950 border border-red-900/50 rounded-lg text-sm flex
items-center gap-2 transition">
            <i class="fa-solid fa-arrow-left"></i> Kembali
        </a>
    </div>

    <!-- Kotak Rincian Buku -->
    <div class="bg-[#1a080c] border border-red-950/40 rounded-2xl p-6 flex flex-col md:flex-row gap-8
shadow-xl">
        <!-- Cover Besar -->
        <div class="w-full md:w-1/3 max-w-xs flex-shrink-0">
            @if($book->cover)
                <img src="{{ asset('storage/' . $book->cover) }}" alt="{{ $book->title }}"
                     class="w-full rounded-xl object-cover shadow-2xl border border-red-950/50">
            @else
                <div class="w-full h-80 bg-gray-900 rounded-xl flex items-center justify-center text-gray-500">
                    Cover Kosong
                </div>
            @endif
        </div>

        <!-- Tabel Rincian Data -->
        <div class="flex-1">
            <table class="w-full text-left border-collapse">
                <tbody>
                    <tr class="border-b border-red-950/40">
                        <th class="py-3.5 px-4 font-semibold text-gray-300 w-44">Judul</th>
                        <td class="py-3.5 px-4 text-white font-medium">{{ $book->title }}</td>
                    </tr>
                    <tr class="border-b border-red-950/40">
                        <th class="py-3.5 px-4 font-semibold text-gray-300">Penulis</th>
                        <td class="py-3.5 px-4 text-white">{{ $book->author }}</td>
                    </tr>
                    <tr class="border-b border-red-950/40">
                        <th class="py-3.5 px-4 font-semibold text-gray-300">Penerbit</th>
                        <td class="py-3.5 px-4 text-white">{{ $book->publisher }}</td>
                    </tr>
                    <tr class="border-b border-red-950/40">
                        <th class="py-3.5 px-4 font-semibold text-gray-300">Tahun Terbit</th>
                        <td class="py-3.5 px-4 text-white">{{ $book->year }}</td>
                    </tr>
                    <tr>
                        <th class="py-3.5 px-4 font-semibold text-gray-300">Kategori</th>
                        <td class="py-3.5 px-4">
```

```
                    <span class="inline-block bg-[#e02438] text-xs font-semibold px-3 py-1 rounded-full text-white">
                        {{ $book->category }}
                    </span>
                </td>
            </tr>
        </tbody>
    </table>
</div>
</div>
</div>
@endsection
```

9.4 Formulir Tambah Buku Baru (resources/views/books/create.blade.php)

Buat berkas create.blade.php di resources/views/books/. Pastikan atribut enctype='multipart/form-data' terpasang pada elemen form agar browser dapat mengirimkan berkas biner gambar ke server.

Kode 9.4: Form Tambah Buku resources/views/books/create.blade.php

```
@extends('layouts.app')

@section('content')
<div class="max-w-2xl mx-auto space-y-6">
  <div class="flex items-center justify-between">
    <h2 class="text-2xl font-bold">Tambah Buku Baru</h2>
    <a href="{{ route('books.index') }}" class="px-4 py-2 bg-[#2a1015] hover:bg-red-950 border border-red-900/50 rounded-lg text-sm">Kembali</a>
  </div>

  <form action="{{ route('books.store') }}" method="POST" enctype="multipart/form-data"
      class="bg-[#1a080c] border border-red-950/40 p-6 rounded-2xl space-y-4 shadow-xl">
    @csrf
    <div>
      <label class="block text-sm font-semibold mb-1 text-gray-300">Judul Buku</label>
      <input type="text" name="title" required
          class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
    </div>
    <div class="grid grid-cols-2 gap-4">
      <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Penulis</label>
        <input type="text" name="author" required
            class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
```

```
        </div>
        <div>
            <label class="block text-sm font-semibold mb-1 text-gray-300">Penerbit</label>
            <input type="text" name="publisher" required
                class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white
focus:outline-none">
        </div>
    </div>
    <div class="grid grid-cols-2 gap-4">
        <div>
            <label class="block text-sm font-semibold mb-1 text-gray-300">Tahun Terbit</label>
            <input type="number" name="year" required
                class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white
focus:outline-none">
        </div>
        <div>
            <label class="block text-sm font-semibold mb-1 text-gray-300">Kategori</label>
            <input type="text" name="category" placeholder="cth: Misteri, Horor, Thriller" required
                class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white
focus:outline-none">
        </div>
    </div>
    <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Unggah Gambar Cover</label>
        <input type="file" name="cover" accept="image/*"
            class="w-full px-3 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-sm text-gray-
300">
    </div>
    <button type="submit"
        class="w-full py-2.5 bg-[#e02438] hover:bg-red-700 font-bold rounded-lg text-white mt-4
transition">
        Simpan Buku
    </button>
  </form>
</div>
@endsection
```

9.5 Formulir Pengubahan Data Buku (resources/views/books/edit.blade.php)

Buat berkas edit.blade.php di resources/views/books/. Mengingat formulir HTML standar hanya mendukung metode GET dan POST, kita menyematkan direktif Blade @method('PUT') agar permintaan dikirimkan sebagai HTTP PUT ke controller.

Kode 9.5: Form Edit Buku resources/views/books/edit.blade.php

```
@extends('layouts.app')
```

```
@section('content')
<div class="max-w-2xl mx-auto space-y-6">
  <div class="flex items-center justify-between">
    <h2 class="text-2xl font-bold">Edit Informasi Buku</h2>
    <a href="{{ route('books.index') }}" class="px-4 py-2 bg-[#2a1015] hover:bg-red-950 border border-red-900/50 rounded-lg text-sm">Kembali</a>
  </div>

  <form action="{{ route('books.update', $book) }}" method="POST" enctype="multipart/form-data"
      class="bg-[#1a080c] border border-red-950/40 p-6 rounded-2xl space-y-4 shadow-xl">
    @csrf
    @method('PUT')
    <div>
      <label class="block text-sm font-semibold mb-1 text-gray-300">Judul Buku</label>
      <input type="text" name="title" value="{{ $book->title }}" required
          class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
    </div>
    <div class="grid grid-cols-2 gap-4">
      <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Penulis</label>
        <input type="text" name="author" value="{{ $book->author }}" required
            class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
      </div>
      <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Penerbit</label>
        <input type="text" name="publisher" value="{{ $book->publisher }}" required
            class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
      </div>
    </div>
    <div class="grid grid-cols-2 gap-4">
      <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Tahun Terbit</label>
        <input type="number" name="year" value="{{ $book->year }}" required
            class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
      </div>
      <div>
        <label class="block text-sm font-semibold mb-1 text-gray-300">Kategori</label>
        <input type="text" name="category" value="{{ $book->category }}" required
            class="w-full px-4 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-white focus:outline-none">
      </div>
    </div>
    <div>
```

```
        <label class="block text-sm font-semibold mb-1 text-gray-300">Ganti Gambar Cover
(Opsional)</label>
        @if($book->cover)
            <p class="text-xs text-gray-400 mb-2">Cover saat ini terpasang di sistem.</p>
        @endif
        <input type="file" name="cover" accept="image/*"
            class="w-full px-3 py-2 bg-[#2a1015] border border-red-900/40 rounded-lg text-sm text-gray-
300">
    </div>
    <button type="submit"
        class="w-full py-2.5 bg-[#f5b301] hover:bg-amber-500 font-bold rounded-lg text-black mt-4
transition">
        Perbarui Buku
    </button>
  </form>
</div>
@endsection
```

12. Langkah 10: Menjalankan Server Pengembangan Lokal dan Pengujian

Seluruh konfigurasi kode, migrasi basis data, controller, dan antarmuka telah lengkap. Sekarang saatnya menyalakan server pengembangan bawaan Laravel:

Perintah 10.1: Menjalankan Server Pengembangan Lokal

```
php artisan serve
```

Setelah server aktif, terminal akan menampilkan alamat http://127.0.0.1:8000. Buka tautan tersebut pada web browser Anda.

Skenario Pengujian Operasional:

- **1. Uji Tambah Data & Upload:** Klik tombol '+ Tambah Buku', isi formulir (misalnya Judul: 'The Devotion of Suspect X', Penulis: 'Keigo Higashino', Penerbit: 'PT Gramedia Pustaka Utama', Tahun: 2016, Kategori: 'Crime, Mystery, Thriller'), pilih file gambar cover, lalu simpan. Periksa apakah kartu buku muncul pada daftar.
- **2. Uji Detail Data:** Klik gambar cover buku pada katalog. Sistem akan membuka rute detail (/books/{id}) yang menyajikan cover besar dan metadata lengkap.
- **3. Uji Fitur Pencarian:** Masukkan kata kunci seperti 'Keigo', 'Gramedia', atau '2016' pada kotak input pencarian di bagian atas, lalu tekan enter. Daftar buku akan otomatis tersaring sesuai kata kunci.

- **4. Uji Pengubahan Data:** Klik tombol kuning berlambang pensil pada salah satu buku, ganti salah satu informasi atau unggah gambar cover baru, lalu klik perbarui. Pastikan data terupdate di database dan gambar baru menggantikan gambar lama.
- **5. Uji Penghapusan Data:** Klik tombol merah berlambang tempat sampah, konfirmasi kotak dialog browser. Pastikan entitas buku terhapus dari daftar dan file cover fisik terhapus dari folder storage.

13. Panduan Pemecahan Masalah (Troubleshooting)

Berikut adalah solusi atas kendala-kendala yang paling sering ditemui selama pengembangan aplikasi Laravel:

13.1 Gambar Cover Rusak / Tampilan Gambar Tidak Muncul (Error 404)

- **Solusi 1:** Periksa apakah perintah 'php artisan storage:link' sudah pernah dijalankan. Jika belum, segera jalankan di terminal proyek.
- **Solusi 2:** Pastikan berkas .env Anda memiliki nilai APP_URL=http://127.0.0.1:8000 yang sesuai dengan port server Anda saat ini.
- **Solusi 3:** Pada Windows, jika tautan simbolis terputus, hapus folder 'public/storage' secara manual lalu jalankan ulang perintah 'php artisan storage:link' menggunakan terminal dengan hak akses administrator.

13.2 Pesan Kesalahan: 'Access Denied for User root@localhost'

- **Solusi 1:** Pastikan layanan MySQL di XAMPP / Laragon berstatus 'Running' (aktif).
- **Solusi 2:** Buka berkas .env, pastikan DB_USERNAME=root dan DB_PASSWORD dikosongkan jika menggunakan pengaturan bawaan XAMPP.
- **Solusi 3:** Jalankan 'php artisan config:clear' di terminal untuk menghapus cache konfigurasi yang tersimpan.

13.3 Kesalahan 'The cover field must not be greater than 2048 kilobytes'

- **Solusi:** Validasi di BookController membatasi ukuran cover maksimal 2 MB (2048 KB). Jika ingin mengunggah cover berukuran lebih besar, ubah aturan 'max:2048' menjadi 'max:5120' (5 MB).
- **Catatan php.ini:** Pastikan pengaturan upload_max_filesize dan post_max_size pada berkas php.ini di instalasi PHP Anda disetel ke nilai yang lebih besar atau sama dengan batas tersebut.

14. Kesimpulan

Dengan menyelesaikan seluruh tahapan dalam panduan ini, Anda telah berhasil membangun aplikasi web manajemen perpustakaan modern berbasis Laravel yang kokoh, terstruktur, aman, dan memiliki performa visual interaktif. Seluruh prinsip penting seperti arsitektur MVC (Model-View-Controller), ORM Eloquent, manajemen berkas storage, sanitasi validasi data, serta manipulasi antarmuka telah diintegrasikan secara optimal.